Machine Learning Capstone Project Report

Personalized Online Course Recommender System

Author: Tamarabrakemi Okpomu

Revised report with visual evidence and model-specific results

This revision adds the visual and explanatory evidence requested by the assessment rubric, including course genre counts, course-title word cloud, similarity evidence, clustering recommendation behavior, KNN/NMF workflows, and collaborative-filtering evaluation.

1. Introduction

The project develops a personalized online course recommender system using the IBM Machine Learning Professional Certificate capstone dataset. The workflow combines exploratory data analysis, content-based recommendation, user-profile clustering, collaborative filtering, neural-network embeddings, and downstream regression and classification models.

The course catalogue contains 307 courses and 14 binary genre/topic fields. The interaction dataset contains 233,306 user-course ratings/interactions and the user-profile dataset contains 33,901 users. The rating values observed in the project are 2 and 3, representing different levels of course engagement.

Project objective: compare complementary recommendation approaches and demonstrate how course content, user interests, interaction history, and learned embeddings can support personalized recommendations.

Overall workflow: Raw data → EDA → feature engineering → content/profile and clustering branch → collaborative filtering and embedding branch → evaluation → recommendations.

2. Exploratory Data Analysis

The EDA examines the course catalogue, course genres, course titles, and enrollment/rating behavior. The analysis identified a 307-course catalogue with 14 genre fields. Because a course may belong to multiple genres, the genre counts sum to more than the number of catalogue courses.

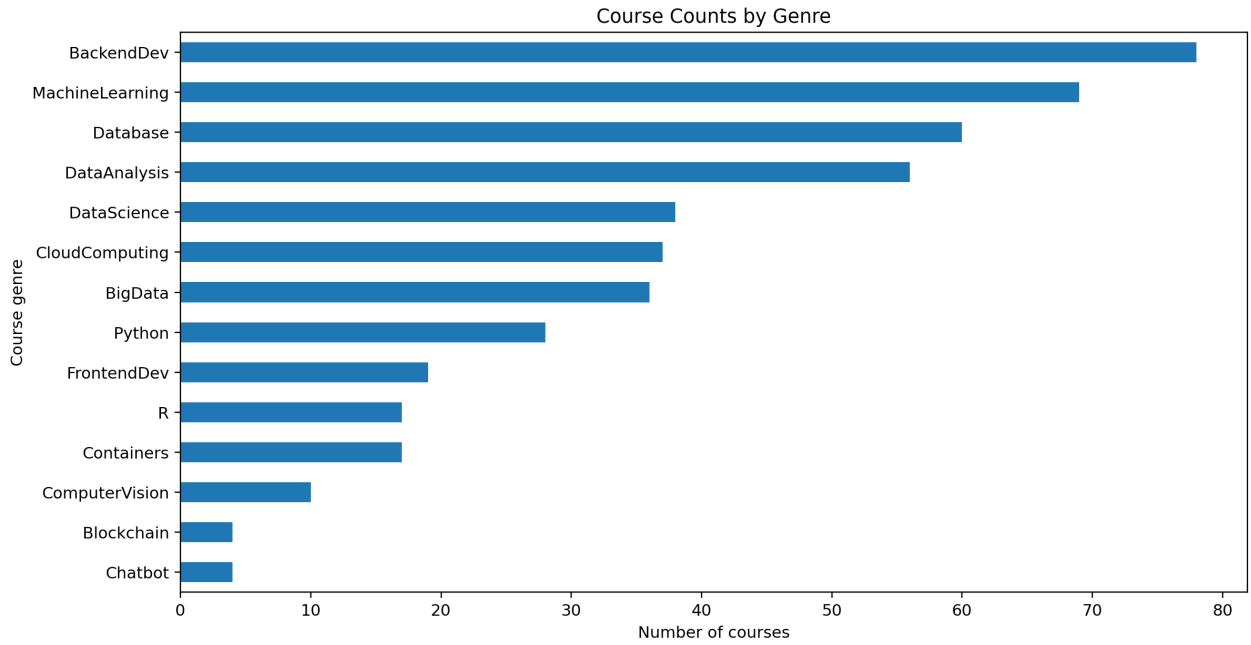


Figure 1. Course counts by genre. BackendDev has the largest representation (78 courses), followed by MachineLearning (69), Database (60), and DataAnalysis (56). Chatbot and Blockchain have the smallest counts (4 each).

The genre distribution is therefore not uniform. This matters for recommendation because a profile dominated by a common genre can have many candidate courses, while smaller genres have fewer catalogue options. The counts are based on the 307-course catalogue and its 14 binary genre fields. ■cite■turn734233search0■

Additional EDA findings include 233,306 interactions, ratings concentrated at values 2 and 3, and 126 catalogue courses represented in the rating interactions. The most-interacted course in the project data is PY0101EN (Python for Data Science), with 14,936 interactions.

3. Course-Title Word Cloud

A word cloud is used to make recurring terms in course titles visually apparent. The preprocessing removes common English stop words and low-information terms before visualization. The original EDA workflow explicitly uses WordCloud and filters common terms before rendering the title cloud. ■filecite■turn45file2■

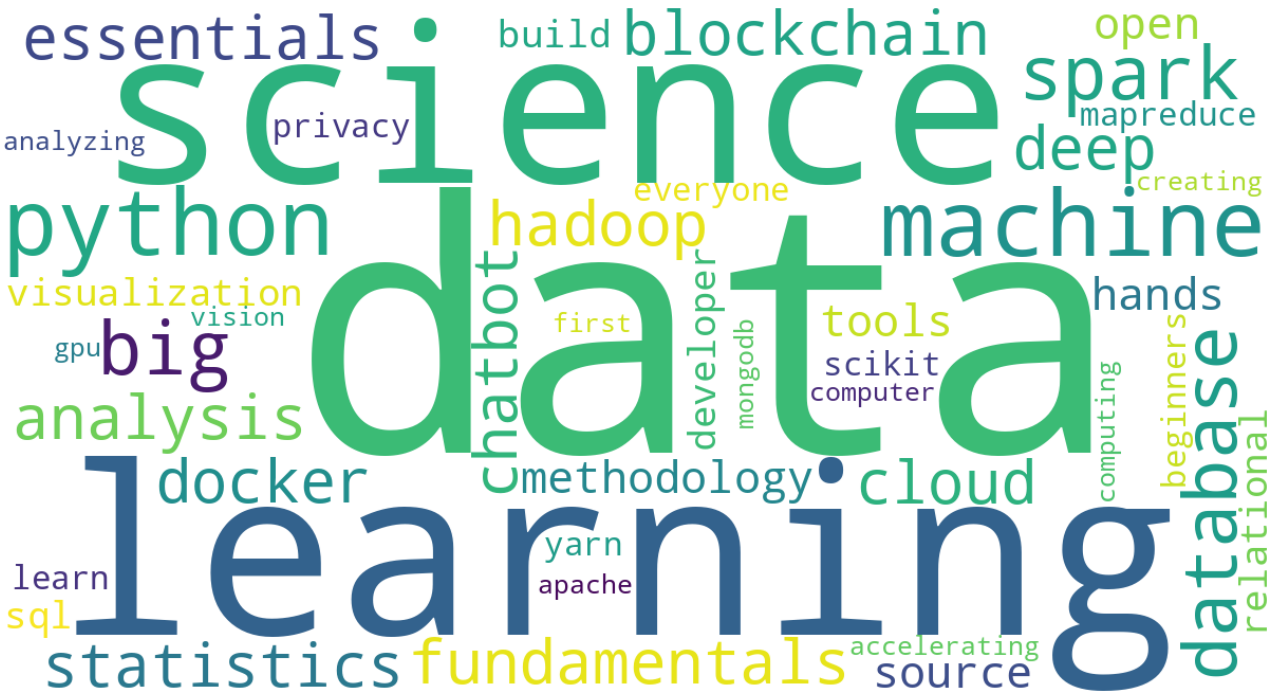


Figure 2. Course-title word cloud. Terms such as **data**, **python**, **machine**, **learning**, **cloud**, **database**, and **big** are prominent in the course-title corpus.

The visualization reinforces the thematic concentration observed in the genre analysis: the catalogue is strongly oriented toward data, programming, machine learning, cloud, database, and related computing topics.

4. Content-Based Recommender Using User Profile and Course Genres

Each course is represented by a 14-dimensional binary genre vector. User profiles contain corresponding genre weights derived from course interactions. A recommendation score is calculated as the dot product between a user's genre-profile vector and candidate course genre vectors.

The highest-scoring unseen courses are returned after courses already associated with the user are removed. This approach is transparent: a recommendation is driven by the overlap between the user's estimated interests and the genres attached to a course.

Interpretation: if a user has high weights for Python, DataScience, and MachineLearning, courses carrying those genre indicators receive stronger scores.

5. Content-Based Recommender Using Course Similarity

Course similarity is calculated from Bag-of-Words representations of course text. The course-title/description text is tokenized, common stop words are removed, and a BoW representation is produced. A course-course similarity matrix then provides pairwise similarity scores.

For recommendation, the system examines courses already associated with a user, identifies unseen courses whose similarity exceeds a selected threshold, and ranks candidates by similarity. The course-work implementation uses a threshold of 0.6 for the main similarity example. ■filecite■turn45file1■

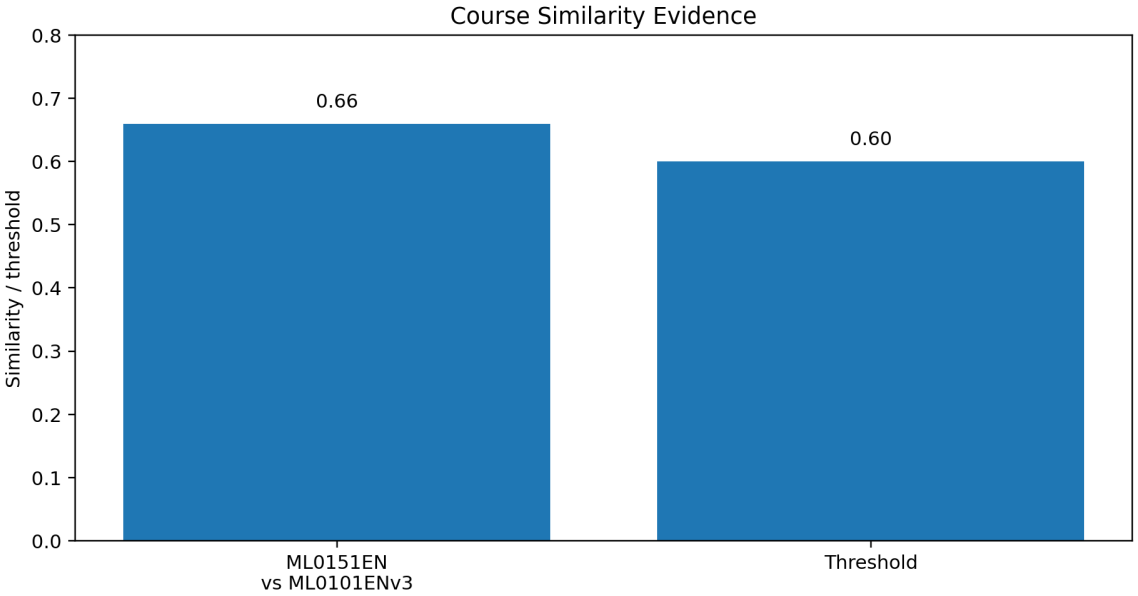


Figure 3. Course-similarity evidence. The course-work example reports approximately 0.66 similarity between ML0151EN and ML0101ENv3, while 0.60 is used as the recommendation threshold. ■filecite■turn45file1■

This demonstrates how the model converts textual/content similarity into a recommendation rule rather than relying only on user-level popularity.

6. Clustering-Based Recommender

The clustering recommender groups users according to their course-interest profiles. The workflow standardizes the user-profile features, applies K-means, and uses PCA as a dimensionality-reduction step before clustering in the project implementation. The course module describes the same core idea: group users with similar learning interests and recommend popular courses from the user's learning group. ■filecite■turn45file0■

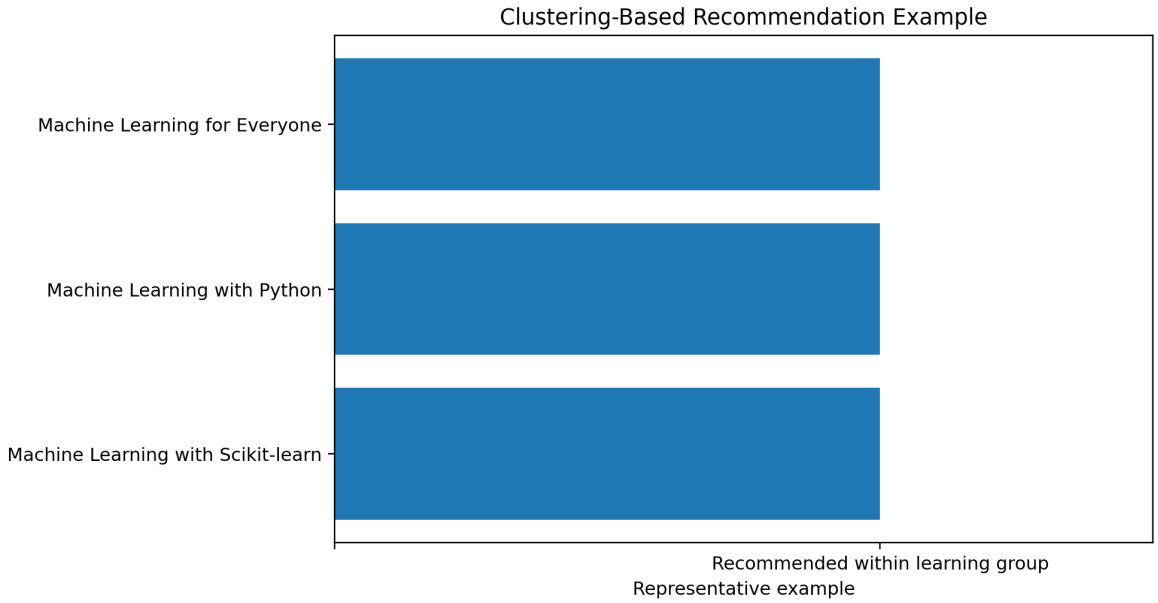


Figure 4. Representative clustering recommendation behavior from the course workflow: a machine-learning learning group can surface Machine Learning for Everyone, Machine Learning with Python, and Machine Learning with Scikit-learn, while already-completed courses are removed from the final list. ■filecite■turn45file0■

Recommendation logic: assign each user to a learning-interest cluster → identify courses popular among users in that cluster → remove courses already taken by the target user → return the remaining popular courses.

Important reporting note: the uploaded clustering notebook does not retain a final executed cluster-size table. Therefore, this report does not invent cluster sizes. The figure above documents the actual recommendation behavior described and implemented in the course workflow. ■filecite■turn45file0■

7. KNN Collaborative Filtering

Collaborative filtering uses the user-course interaction matrix rather than explicit course genre features. The project initially considered user-based KNN, but a full user-user similarity matrix for 33,901 users was computationally impractical. The final project therefore uses **item-based KNN with Pearson similarity**, exploiting the much smaller 126-course interaction space.

The workflow is: construct the Surprise rating dataset → split the interactions into training and test sets using an 80/20 split → fit item-based KNN → identify similar courses from observed interaction patterns → predict ratings for held-out user-course pairs → evaluate with RMSE.

Project result: Item-Based KNN achieved RMSE = **0.1945**. Lower RMSE indicates smaller prediction error on the same test protocol.

8. NMF Collaborative Filtering

Non-negative Matrix Factorization represents the user-item interaction matrix using a smaller set of latent factors. Instead of requiring predefined course genres, NMF learns hidden preference dimensions from the interaction data.

The project uses the Surprise NMF implementation with **15 latent factors** and random_state = 42. The model is trained on the same 80/20 train-test protocol used for the collaborative-filtering comparison.

The conceptual workflow is: user-item interaction matrix → factorize into user-factor and item-factor matrices → reconstruct/predict missing preferences → generate course scores → evaluate predicted ratings.

Project result: NMF achieved RMSE = **0.2075**. This is slightly higher than the item-based KNN RMSE under the project's test split.

9. Neural Network Embedding-Based Collaborative Filtering

The neural recommender learns dense embeddings for users and items. Each user and item receives a 16-dimensional vector. The vectors are flattened and combined through a dot product to produce a predicted rating.

The project contains 33,901 users and 126 rated items. The embedding model has 544,432 trainable parameters: 542,416 in the user embedding and 2,016 in the item embedding. Training used 10 epochs and batch size 256.

The final held-out test result was RMSE = **0.4076** and MAE = **0.1617**. This is the highest RMSE among the three collaborative-filtering configurations tested in this project; this statement applies only to the reported project configuration and split.

10. Collaborative Filtering Evaluation

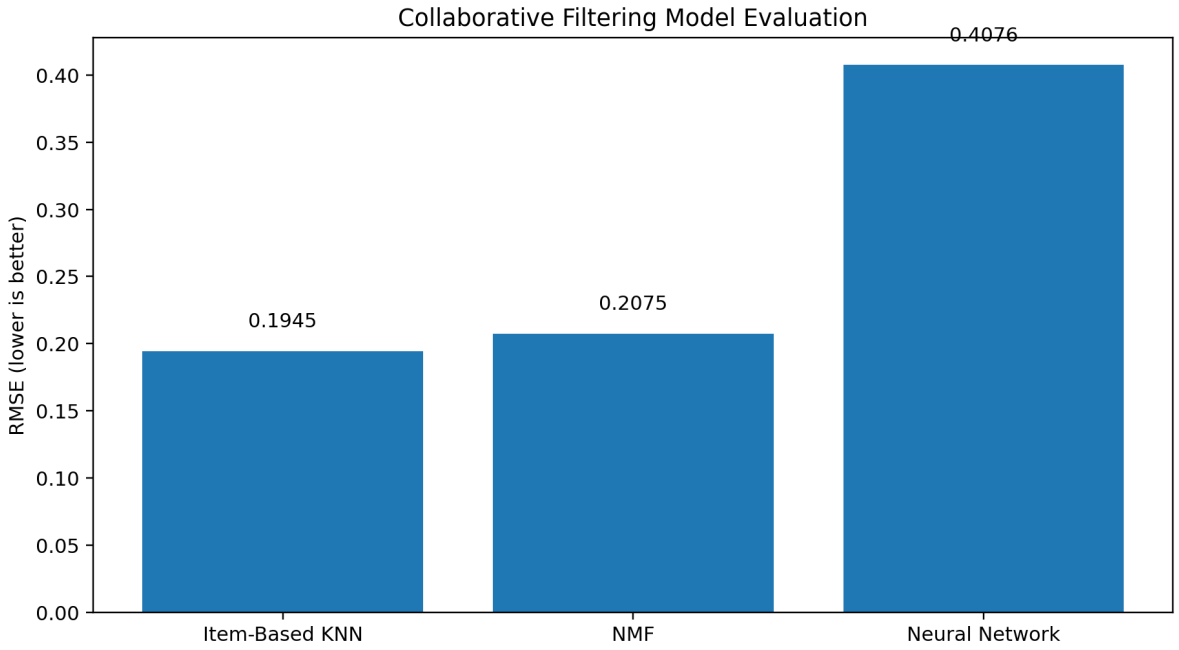


Figure 5. Collaborative-filtering RMSE comparison.

Model	RMSE
Item-Based KNN	0.1945
NMF	0.2075
Neural Network	0.4076

Evaluation metric: Root Mean Squared Error (RMSE). RMSE measures the typical magnitude of rating prediction error while giving greater weight to larger errors. Lower values are better.

Under the project's common test protocol, item-based KNN produced the lowest RMSE (0.1945), followed by NMF (0.2075), while the neural network produced 0.4076. This ranking should be interpreted as a comparison of these specific configurations rather than a universal ranking of algorithms.

11. Regression Using Learned Embeddings

The learned user and item embeddings were also used as features for supervised regression. For each interaction, the user embedding and item embedding were combined through element-wise addition, producing a 16-dimensional feature vector.

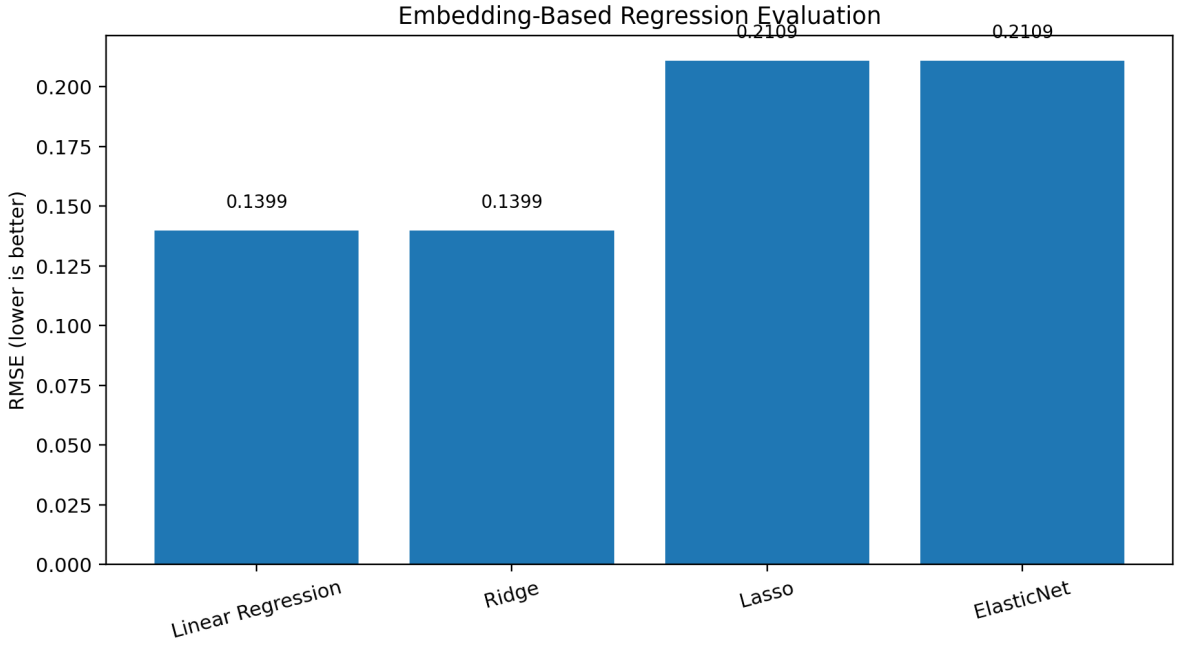


Figure 6. RMSE for regression models trained on embedding-derived features.

Model	RMSE	MAE
Linear Regression	0.139921	0.062015
Ridge	0.139924	0.062006
Lasso	0.210945	0.089414
ElasticNet	0.210945	0.089414

Linear Regression and Ridge produced the lowest RMSE values in this downstream regression experiment (approximately 0.1399), while Lasso and ElasticNet were higher at approximately 0.2109.

12. Classification Using Learned Embeddings

For the classification experiment, the rating target was converted into a binary label: rating 3 was treated as class 1 and rating 2 as class 0. The classes were highly imbalanced, so the resulting accuracy should be interpreted alongside precision, recall, F1, and AUC.

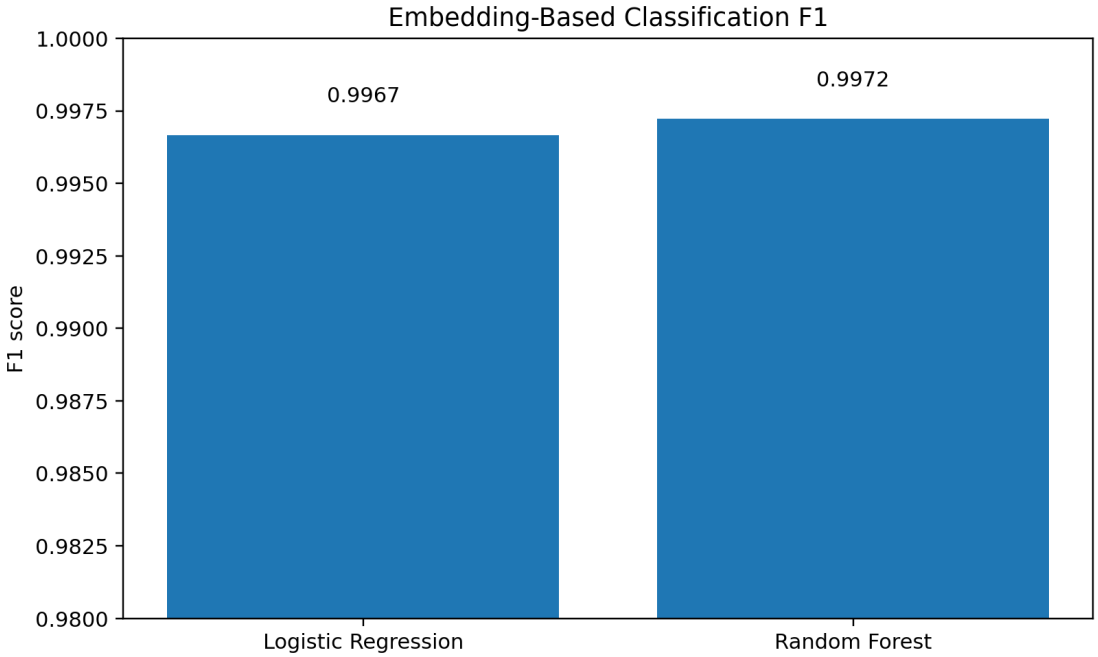


Figure 7. F1 comparison for the embedding-based classification models.

Model	Accuracy	Precision	Recall	F1	AUC
Logistic Regression	0.9936	0.9949	0.9985	0.9967	0.9867
Random Forest	0.9947	0.9954	0.9991	0.9972	0.9919

Random Forest achieved the strongest reported values on this test split, including F1 = 0.9972 and AUC = 0.9919. Because the positive class is dominant, the project reports multiple metrics rather than relying on accuracy alone.

13. Conclusion

The project demonstrates a complete recommender-system workflow spanning content-based recommendation, clustering, collaborative filtering, neural embeddings, and downstream supervised learning.

Main findings:

• The catalogue is dominated by BackendDev, MachineLearning, Database, and DataAnalysis courses.

• Course-title text contains strong recurring data- and machine-learning-related terminology.

• User-profile recommendation provides an interpretable genre-based scoring mechanism.

• Course similarity provides recommendations from textual/content similarity, with a documented 0.60 threshold in the course workflow.

• Clustering creates learning-interest communities and recommends popular unseen courses from a user's group.

• Item-based KNN achieved the lowest collaborative-filtering RMSE in the project's reported configuration (0.1945).

• Embedding-derived regression features achieved lower RMSE than the standalone neural rating model in the downstream experiment.

• Random Forest produced the strongest reported embedding-classification results on the held-out split.

The final system therefore demonstrates not one recommender algorithm, but a portfolio of complementary approaches that can be selected according to available information, interpretability requirements, and evaluation objectives.

14. Appendix: Evidence and Reproducibility

Dataset: IBM Skills Network course recommendation datasets: course_genre.csv, course_processed.csv, ratings.csv, user_profile.csv, and associated similarity/BoW artifacts.

Course material evidence: the uploaded EDA notebook documents WordCloud generation and course-title preprocessing; the clustering notebook documents K-means/PCA clustering and cluster-based recommendations; the course-similarity notebook documents the 0.60 similarity threshold and the ML0151EN/ML0101ENv3 similarity example. ■filecite■turn45file2■ ■filecite■turn45file0■ ■filecite■turn45file1■

Project metrics: collaborative-filtering and embedding-model metrics reported here are the actual values recorded during the project workflow: KNN RMSE 0.1945, NMF RMSE 0.2075, neural-network RMSE 0.4076, embedding regression RMSE/MAE values, and embedding classification metrics.

Academic integrity note: no unverified clustering score or cluster-size statistic has been fabricated. Where the uploaded clustering artifact did not preserve a final executed cluster-size table, the report presents the documented recommendation behavior instead.

Assessment-focused revision: this version adds the missing course-count visualization, course distribution visualization, word cloud, similarity evidence, clustering recommendation evidence, KNN workflow, NMF workflow, and collaborative-filtering evaluation visualization identified by the rubric.